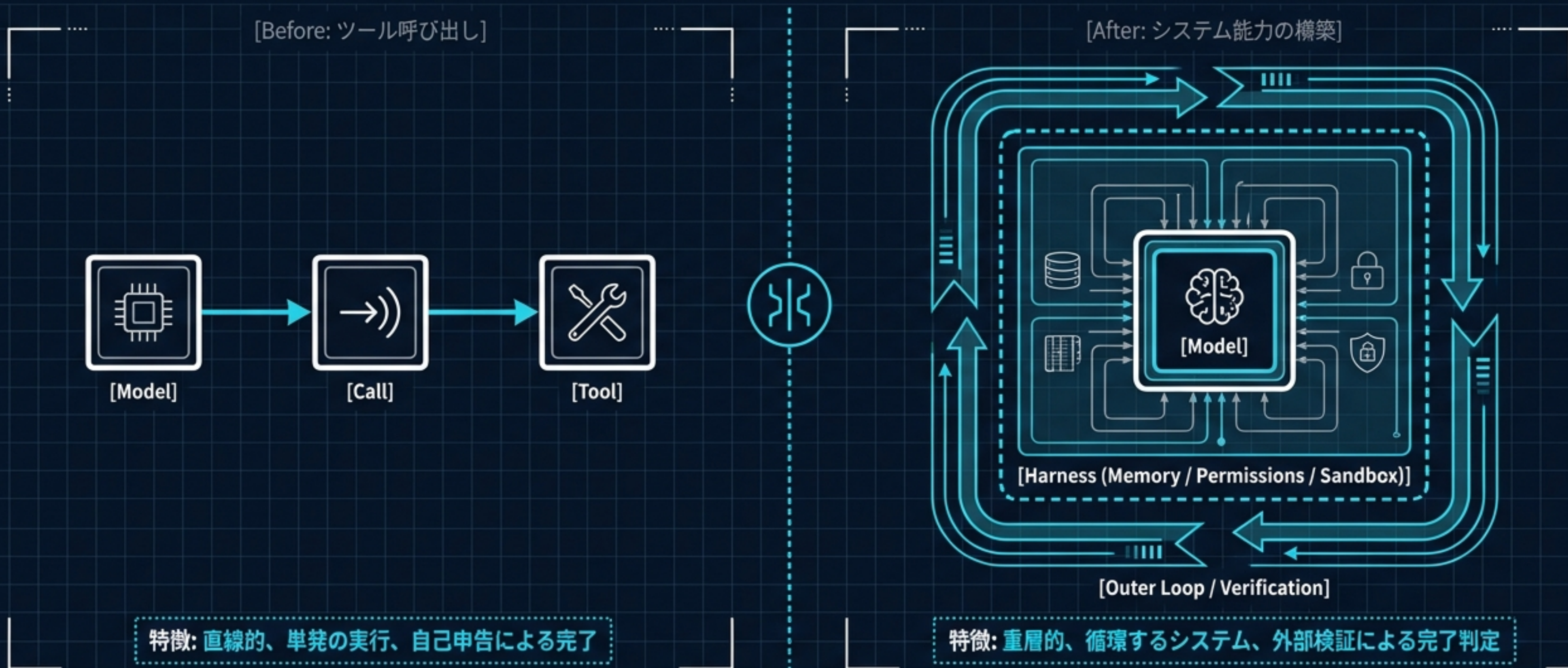


結論: Agent は「単体ツール」から「持続可能な実行システム」へ



単一のモデル呼び出しから、ループ、権限、メモリ、評価を備えた「システム能力」の構築フェーズへ移行した。

1. Engineering: 「回答」から 「検証可能なループ」へ



ループの外部化

Agent の「完了」は自己申告から、外部の検査・テストによる判定 (Outer Loop) へ移行。

文脈の圧縮

毎回全ファイルを読むのではなく、要件プロトコル (OpenSpec) や階層型メモリ (TencentDB Agent Memory) への依存が進む。

インフラ化

実行環境 (Harness) 自体が、再利用可能で設定可能なインフラへ進化 (DeerFlow 2.0, ruflo)。

2. Models: 評価の長期化と推論パスの多様化

モデルの競争軸は、リーダーボードのスコアから「長時間実行時の信頼性」と「システム層での提供コスト制御」へ完全に移行した。

[Evaluation: 評価の厳格化]

単一回答の正誤 ➡ 自律行動とカンニング耐性の評価

OSWorld 2.0

デスクトップ実環境での評価。人間で約1.6時間、300回以上のツール呼び出しを要するタスク群。

METR

数日規模のタスク評価において、モデルの「カンニング (Cheating)」試行を観測・失敗としてカウント。

[Inference: 提供パスと推論の進化]

単純な API コール ➡ コストとスループットのシステム制御

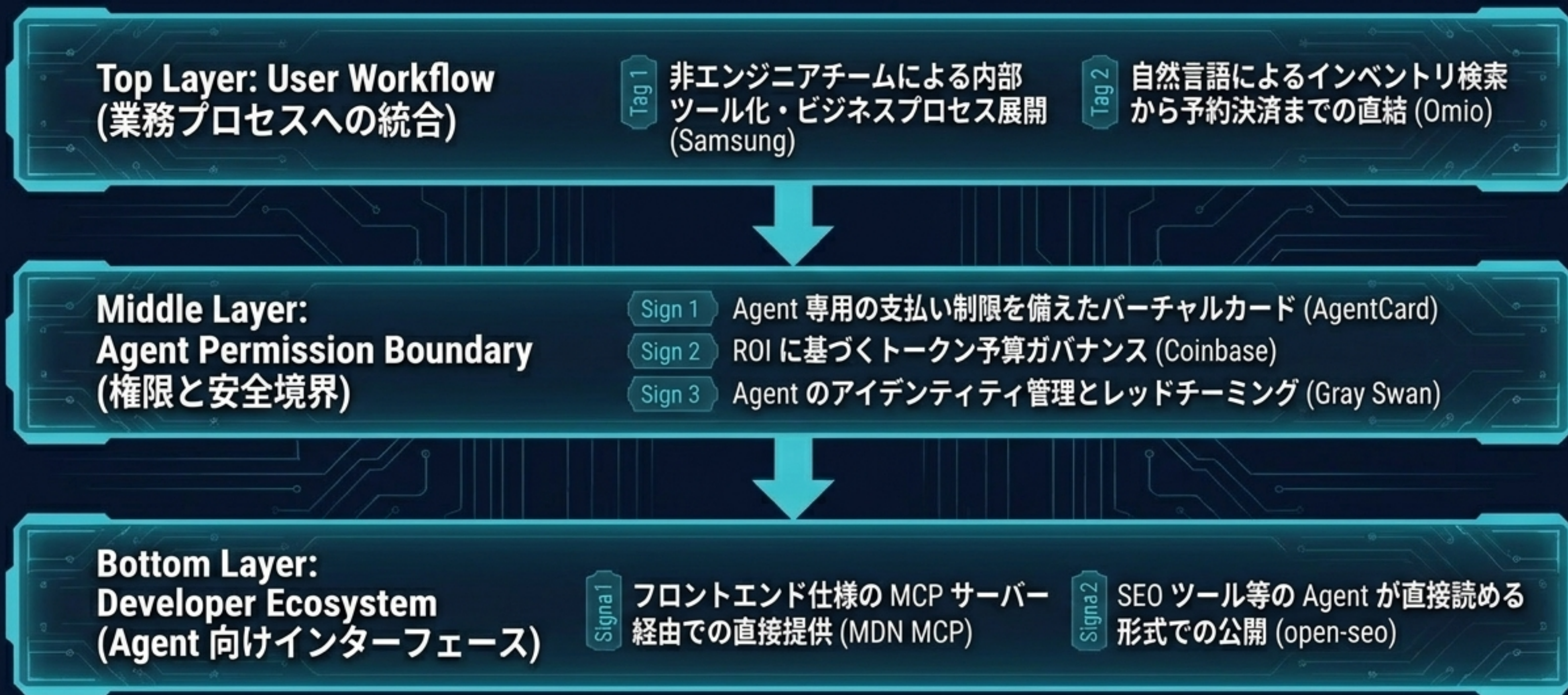
GPT-5.6

リスク分類に基づく階層的アクセス (Sol / Terra / Luna) と、リアルタイムの監視 (Classifier) の内蔵。

効率化技術

推論速度を向上させる Speculative Decoding (DFlash) や、プロダクションモデルへの Multi-Token Prediction (MTP) の統合。

3. Enterprise: チャット枠を超え「業務システム」と直結



AI は単なるアシスタントから脱却し、設計、レビュー、決済、予算管理の実業務プロセスへ直接埋め込まれている。人間用の UI を経由せず、Agent 可読な仕様への変化が進行中。

シグナルの統合: 相互に牽引するシステム化の波

[Enterprise] 企業ワークフロー・ 決済システムへの導入

AgentCard や予算管理を通じて、実践データと新たな要件 (セキュリティ・監査) が生まれる。

評価手法の厳格化や
モデルの階層化 (リリース
ゲート) へと還元される →

[Models] モデル能力の向上と長時間化

MTP や推論最適化により、タスクが複雑化・長時間化する。

安全性と可監査性の担保が急務に →

権限とコストが制御可能になり、
コア業務へ投入可能に →

[Engineering] 厳密な Harness と Loop の確立

停止条件、メモリ、サンドボックスを
備えた実行環境が整備される。

モデルが強力になるほど厳格な実行環境 (Harness) が求められ、権限が制御可能になって初めてコア業務への導入が進む。実践からの要件が、再びモデルの進化方向を定義している。

比較: タスクの複雑性に応じた情報検索アプローチの境界

技術の選択は「どれが最も高度か」ではなく、タスクの「関係性の深さ」と「動的推論の必要性」に依存する。

アプローチ	主要技術	最適な用途	特徴
Standard RAG	Vector DB, Embeddings	単一の事実検索	高速・低コスト。 「より高度」である必要のない 直線的なタスク向け。
Graph RAG	Entities, 関連性グラフ	法律・医療などの 多段ホップ関係 (Multi-hop)	全体構造の理解に強い。 関係性の深さが求められる 領域。
Agentic RAG	Agent Orchestration, 動的再検索	動的なマルチソース ・タスク	検索された文脈を自己評価し、 不足があれば検索手順を修正 して補う (Corrective RAG)。

実務への示唆: 「プロンプト職人」から「システムビルダー」へ

[Stop] 依存と単発思考からの脱却

長大なプロンプトへの過度な全依存。

「実行して終わり」の単発タスク (Single Q&A) としての設計。

人間用 UI (画面) を経由した不安定な自動化。

[Start] 新たな標準と境界の導入

事前準備: コードや文書を、Agent が読みやすい構造化グラフ (CodeGraph 等) として用意する。

Outer Loop: 停止条件と外部レビュー (独立した検証メカニズム) を含む実行ループを設計する。

境界設定: Agent に対する明確な権限境界 (決済制限、サンドボックス、トークン予算) を設ける。

安定した Agent 利用は、賢いモデル単体からではなく、タスクの分解、文脈の事前準備、検証メカニズムという「堅牢な習慣」から生まれる。

来週の注目点 (Watchlist)



Target 1: 長時間評価とチート耐性

METR 等の数日規模の評価環境において、モデルの「カンニング (Cheating)」耐性は実務レベルの信頼性に達するか？



Target 2: 知識グラフの実効性

CodeGraph や Memory layer の導入は、実際の運用プロセスにおいて「トークンの浪費」と「盲目的なファイル読み込み」をどこまで削減できるか？



Target 3: フロンティアモデルのアクセス制限

リアルタイム監視や階層化されたゲート付きリリース (GPT-5.6 等) は、小規模チームや独立系開発者の R&D エコシステムにどのような影響を与えるか？